

How Instagram Scaled to 2.5 Billion Users

#49: Break Into Instagram Scalability Techniques (7 minutes)



NEO KIM

JUN 11, 2024  PAID

Get my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post outlines how Instagram scaled its infrastructure. If you want to learn more, find references at the bottom of the page.

- *Share this post* & I'll send you some rewards for the referrals.

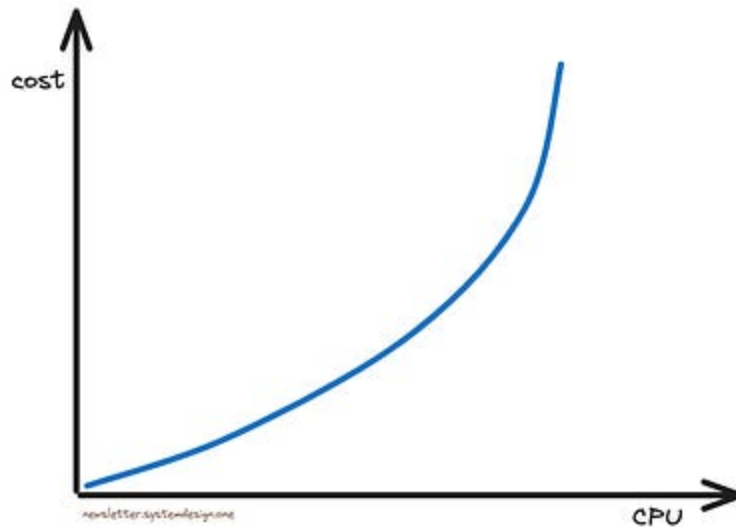
Note: This post is based on my research and may differ from real-world implementation.

Once upon a time, 2 Stanford graduates decided to make a location check-in app.

Yet they noticed photo sharing was the most used feature of their app.

So they pivoted to create a photo-sharing app and called it Instagram.

As more users joined in the early days, they bought new hardware to scale.



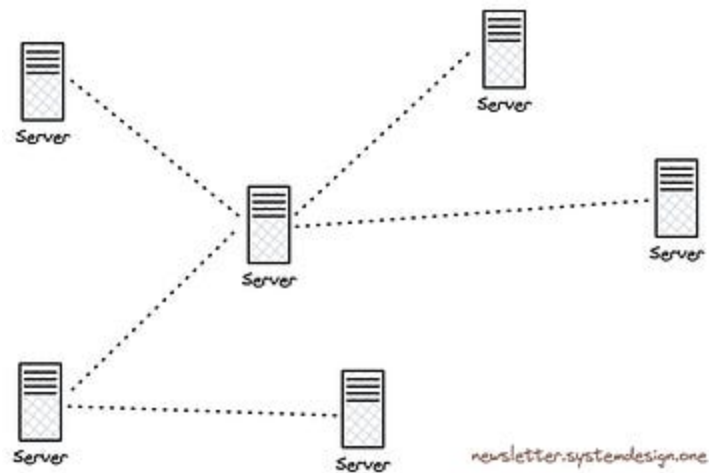
Vertical Scaling

But scaling vertically became expensive after a specific point.

Yet their growth was skyrocketing.

And hit 30 million users in less than 2 years.

So they scaled out by adding new servers.



Horizontal Scaling

Although it temporarily solved their scalability issues, there were newer problems from hypergrowth.

Here are some of them:

1. Resource Usage:

More hardware allowed them to scale out.

Yet each server wasn't used to maximum capacity.

Put simply, resources got wasted as their server throughput wasn't high.

2. Data Consistency:

They installed data centers across the world to handle massive traffic.

And data consistency is needed for a better user experience.

But it became difficult with many data centers.

3. Performance:

They run Python on the application server.

This means they have processes instead of threads due to global interpreter lock (GIL).

So there were some performance limitations.



Instagram Infrastructure

The smart engineers at Instagram used simple ideas to solve these difficult problems.

Here's how they provide extreme scalability:

1. Resource Usage:

Python is beautiful.

But C is faster.

So they replaced stable functions that are extensively used with Cython and C/C++.

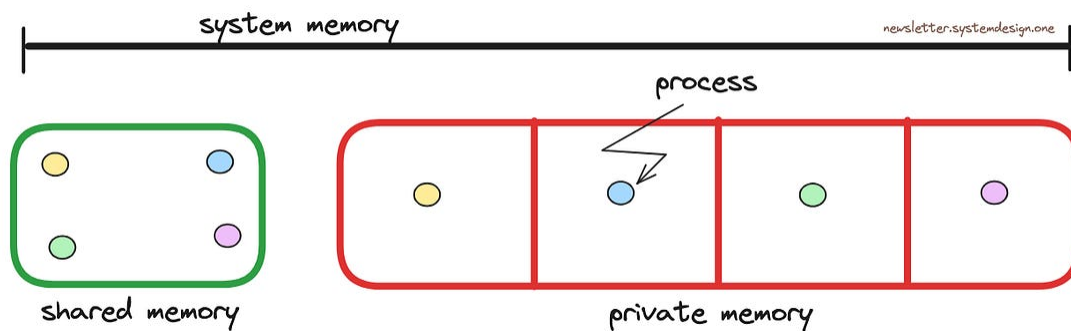
It reduced their CPU usage.

Also they didn't buy expensive hardware to scale up.

Instead reduced CPU instructions needed to finish a task.

This means running good code.

Thus handling more users on each server.



Shared Memory vs Private Memory

Besides the number of Python processes on a server is limited by the system memory available.

While each process has a private memory and access to shared memory.

That means more processes can run if common data objects get moved to shared memory.

So they moved data objects from private memory to shared memory if a single copy was enough.

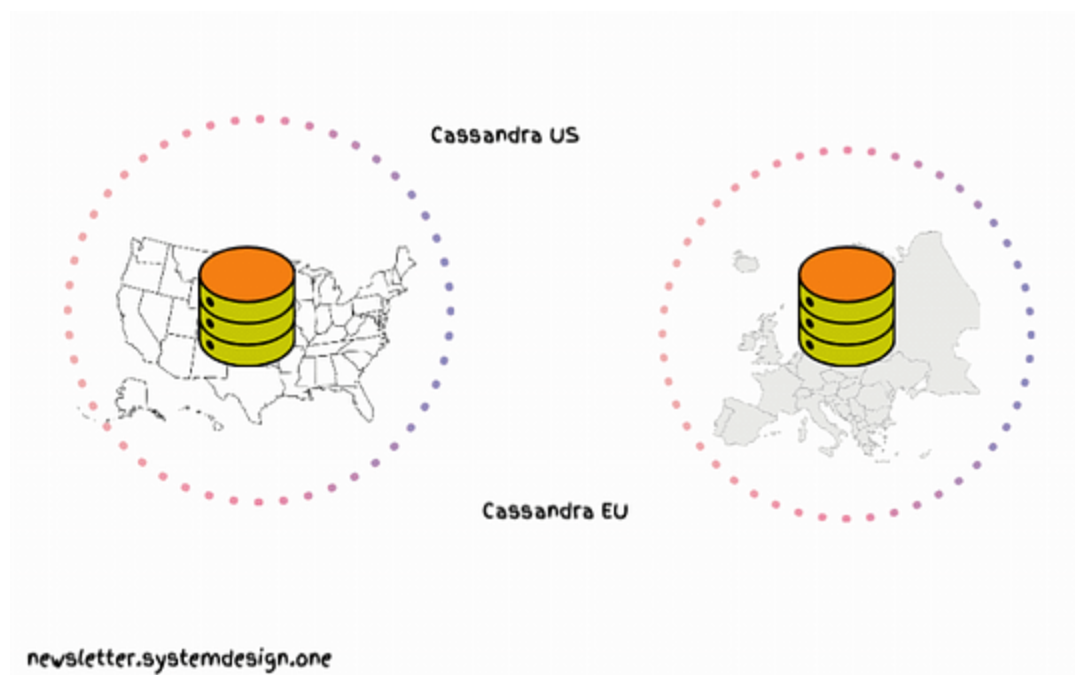
Also removed unused code to reduce the amount of code in memory.

2. Data Consistency:

They use Cassandra to support the Instagram user feed.

Each Cassandra node can be deployed in a different data center and synchronized via eventual consistency.

Yet running Cassandra nodes in different *continents* makes latency worse and data consistency difficult.



Optimal Data Placement With Separate Cassandra Clusters

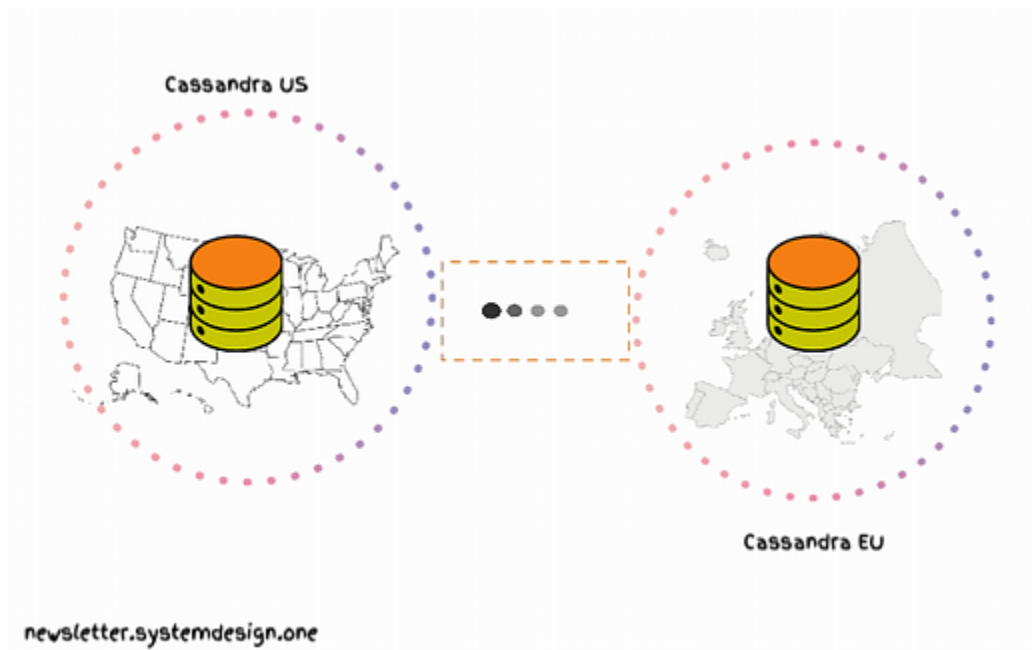
So they don't run a single Cassandra cluster for the whole world.

Instead separate ones for each continent.

This means the Cassandra cluster in Europe will store the data of European users. While the Cassandra cluster in the US will store the data of the US users.

Put another way, the number of replicas isn't equal to the number of data centers.

Yet there's a risk of downtime with separate Cassandra clusters for each region.



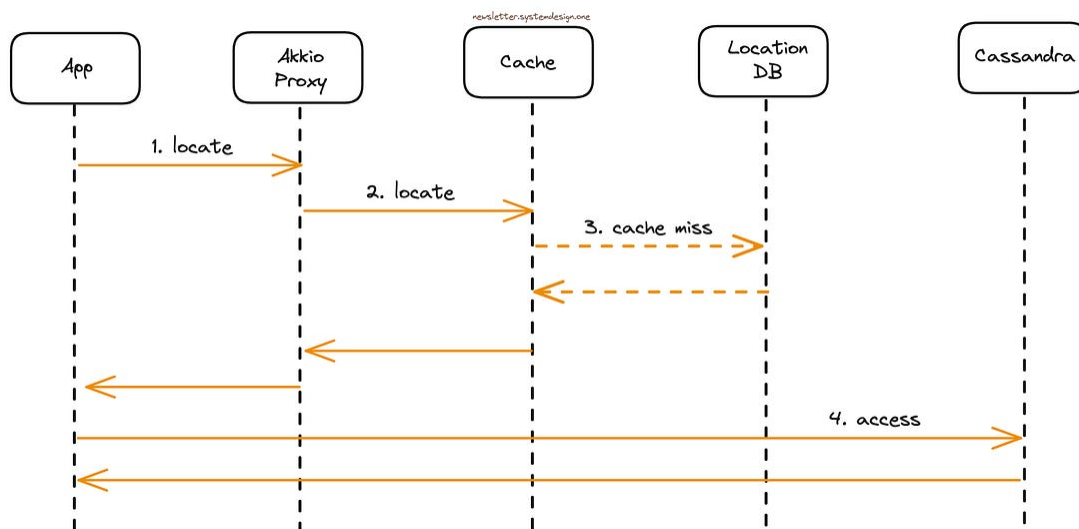
Keeping Single Replica in Another Region for Disaster Readiness

So they keep a single replica in another region and use quorum to route requests with an extra latency.

Besides they use Akkio for optimal data placement. It splits data into logical units with strong locality.

Imagine **Akkio** as a service that knows when and how to move data for low latency.

They route every user request through the global load balancer. And here's how they find the right Cassandra cluster for a request:



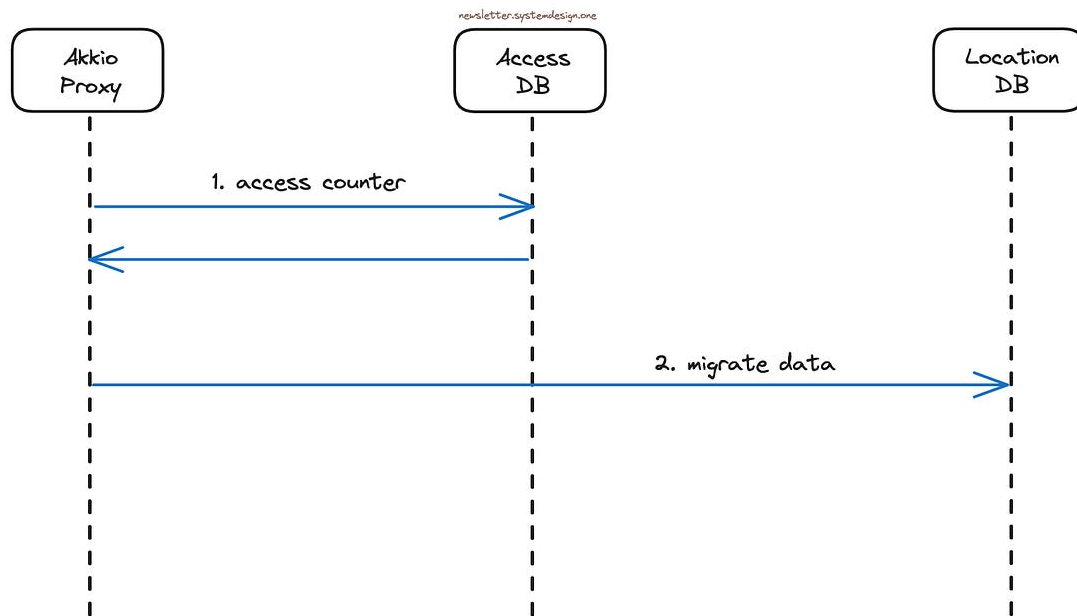
Finding the Correct Cassandra Cluster for a User Request

1. The user sends a request to the app but it doesn't know the user's Cassandra cluster
2. The app forwards the request to the Akkio proxy
3. The Akkio proxy asks the cache layer
4. The Akkio proxy will talk to the location database on a cache miss. It's replicated across every region as it has a small dataset
5. The Akkio proxy returns the user's Cassandra cluster information to the app
6. The app directly accesses the right Cassandra cluster based on the response
7. The app caches the information for future requests

The Akkio requests take around 10 ms. So the extra latency is tolerable.

Also they migrate the user data if a user moves to another continent and settles down there.

And here's how they migrate data if a user moves between continents:

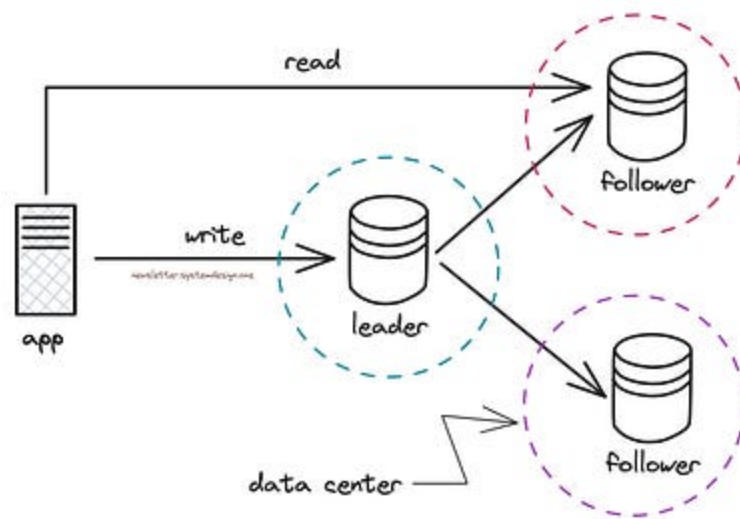


Data Migration When a User Moves to Another Continent

1. The Akkio proxy knows the user's Cassandra cluster location from the generated response
2. They maintain a counter in the access database. It records the number of accesses from a specific region for each user
3. The Akkio proxy increments the access database counter with each request. They could find the exact location of the user based on the IP address
4. Akkio migrates user data to another Cassandra cluster if the counter exceeds a limit

Besides they store user information, friendships, and picture metadata in PostgreSQL.

Yet they get more read requests than write requests.



Scaling Out PostgreSQL

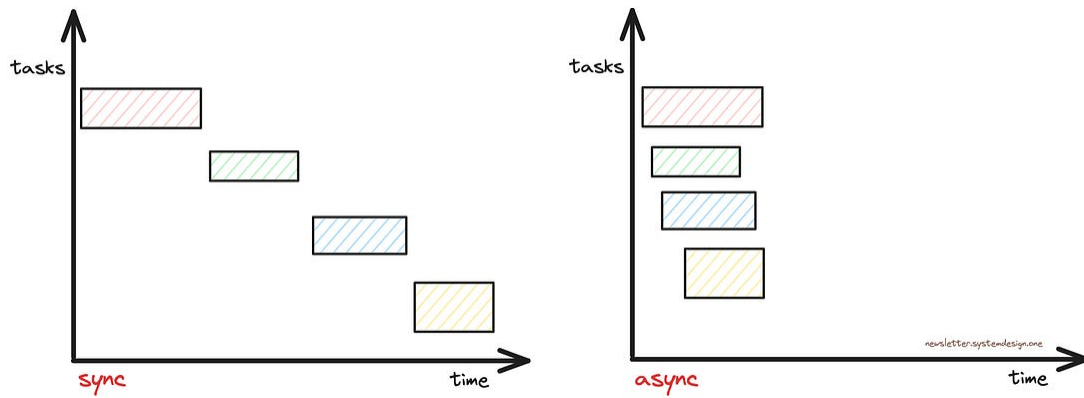
So they run PostgreSQL in leader-follower replication topology.

And route write requests to the leader. While read requests get routed to the follower on the same data center.

3. Performance:

The app layer is *synchronous* and must wait for external services to respond.

That means fewer CPU instructions get executed and server starvation.



Synchronous vs Asynchronous Execution

So they use Python async IO to talk asynchronously with external services.

Also they have Python processes instead of threads.

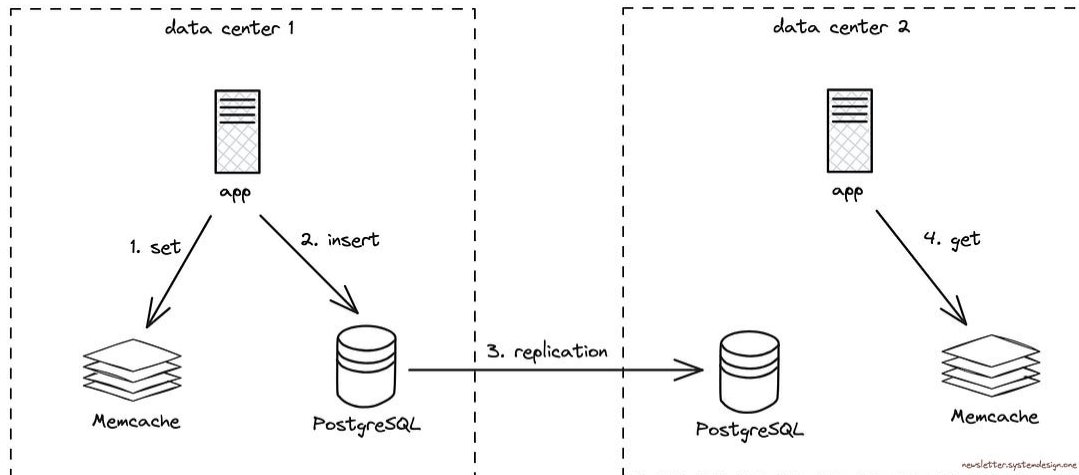
In other words, server capacity is wasted if a request takes longer to finish.

So they terminate a request if it takes more than 12 seconds to finish.

Besides they use Memcache to protect PostgreSQL against massive traffic. And route read requests to it.

It's used as a key-value store and can handle millions of requests per second.

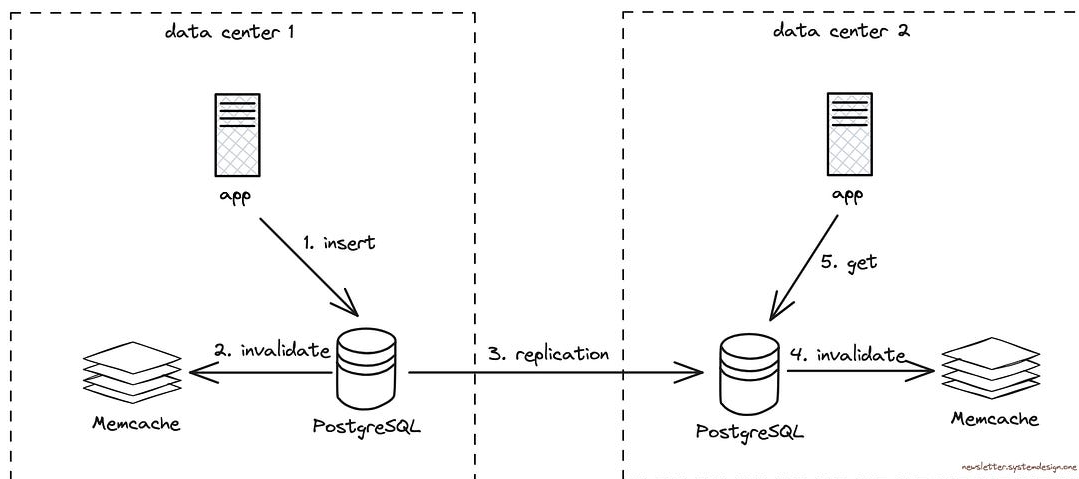
Yet they don't replicate Memcache across data centers for *low latency*.



Stale Data Served by Cache in Another Data Center

Thus there's a risk of stale data.

For example, comments received on a picture in one data center wouldn't be visible to someone viewing the same picture from another data center.



Invalidating Cache on Updates to PostgreSQL

So they don't insert data directly into Memcache.

Instead a separate service watches for PostgreSQL updates and then invalidates Memcache.

That means the first request after cache invalidation gets routed to PostgreSQL.

Yet there's a risk of PostgreSQL overload.

For example, finding the number of likes received on a popular picture could become expensive:

```
select count(*) from user_likes_media where media_id = 42;
```

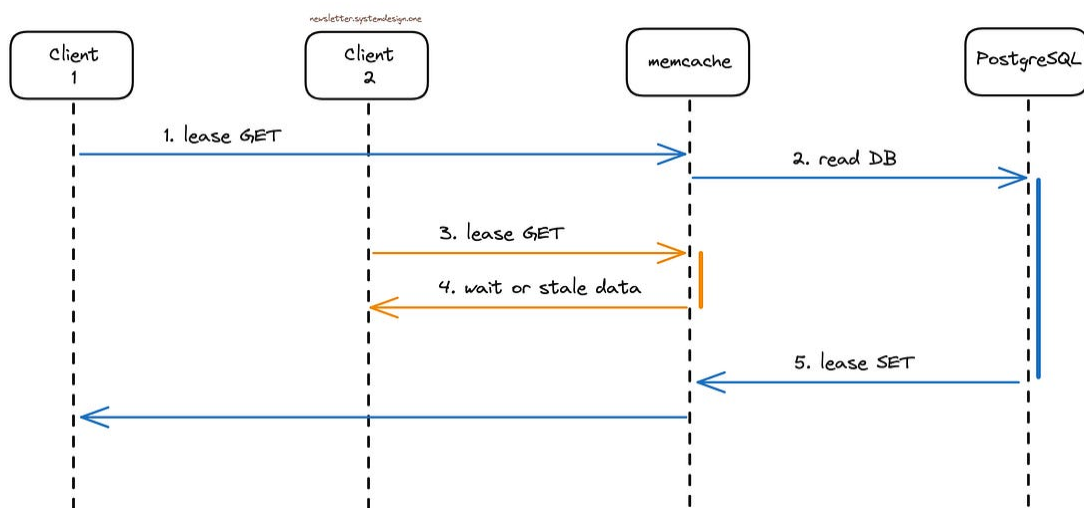
So they create denormalized tables to handle expensive queries:

```
select count from media_likes where media_id = 42;
```

Thus providing faster responses with less resource usage.

But invalidating every cache at once could cause a thundering herd problem on PostgreSQL.

So they use **Memcache lease**.



Memcache Lease Workflow

Here's how it works:

1. The first request gets routed to Memcache. But it isn't a normal GET operation instead *lease* GET operation
 2. The Memcache forwards the request to PostgreSQL and asks the client to wait
 3. The Memcache asks the second request to wait if the first request isn't fulfilled yet
 4. The Memcache offers stale data as an alternative option to the second request. It should be enough in most cases such as finding the number of likes on a picture
 5. The response to the first request updates the Memcache
 6. The pending and newer requests get fresh data from Memcache
-

Scaling an app is a constant effort.

And every architectural decision is a tradeoff.

While Instagram was able to serve 2.5 billion users with these techniques.

This case study shows Python and other proven technologies should be enough to handle internet-scale traffic.

Consider subscribing to get simplified case studies delivered straight to your inbox:

✓ Subscribed



Follow me on [LinkedIn](#) | [YouTube](#) | [Threads](#) | [Twitter](#) |
[Instagram](#)

Don't keep this newsletter a secret. Refer just 3 people and I'll send you some rewards as a thank you:

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share

How YouTube Was Able to Support 2.49 Billion



Users With MySQL

NEO KIM · MAY 31, 2024

[Read full story](#)



How Facebook Scaled Live Video to a Billion Users

NEO KIM · MAY 24, 2024

[Read full story](#)

References

- [Scaling Instagram Infrastructure](#)
- [SREcon19 Asia/Pacific - Cross Continent Infrastructure Scaling at Instagram](#)
- [Open-sourcing a 10x reduction in Apache Cassandra tail latency](#)
- [What Powers Instagram: Hundreds of Instances, Dozens of Technologies](#)
- [Managing data store locality at scale with Akkio](#)
- [Location-Aware Distribution: Configuring servers at scale](#)
- [Burbn - the early Instagram](#)
- [How Instagram is scaling its infrastructure across the ocean](#)
- [How do threads work in Python, and what are common Python-threading-specific pitfalls?](#)
- [Why should I use a thread vs. using a process?](#)
- [What makes C faster than Python?](#)